



United International University (UIU)
Department of Computer Science and Engineering (CSE)
Course: VLSI Design (CSE 4327)

Experiment 8: Physical Design of a 32-bit Adder using Innovus

Objectives

1. To understand the physical design (place-and-route) flow: design import, floorplanning, power planning, placement, clock tree synthesis, routing, and verification.
2. To import the gate-level netlist and SDC constraints produced in Experiment 6 into Cadence Innovus, and to set up the floorplan and power distribution network for the design.
3. To place the standard cells and I/O pins, and to check and optimize pre-CTS (pre-clock-tree) timing.
4. To perform Clock Tree Synthesis (CTS) and verify post-CTS timing.
5. To route the design, verify its physical geometry and connectivity, and generate a GDSII file ready for tape-out.

Introduction

Physical design (place and route) is the stage where a synthesized gate-level netlist is turned into an actual chip layout: standard cells are placed on the die, power and ground rails and a clock distribution network are built, and the cells are wired together with real metal routing - all while continuing to meet the timing constraints used at synthesis. Cadence Innovus is the tool used for this stage, taking as its inputs the gate-level netlist, the SDC timing constraints, the standard-cell timing libraries, and a LEF (Library Exchange Format) file describing the physical geometry of each cell and routing layer.

This experiment continues directly from Experiment 6 (Synthesis of a 32-bit Pipelined Adder using Genus): the gate-level netlist (Adder_32bit_8_stage_netlist.v) and the SDC constraints (Adder_32bit_8_stage.sdc) generated there are carried forward here as the starting point for physical design.

Note: This experiment assumes your Cadence tools environment (cshrc/csh) has already been configured, as covered in the earlier labs. If you are starting a fresh session, remember to switch to csh before invoking any Cadence tool.

Note: The reference screenshots in this manual were captured for a design elaborated as pipelined_32b_adder, the same design used for the Genus screenshots in Experiment 6. Your own hierarchy and title bars will show Adder_32bit_8_stage; the flow and every dialog shown are otherwise identical.

Part 1: Setting Up the Physical Design Directory

1. Create the lab directory and its sub-directories under your work area:

```
mkdir -p lab8/lib lab8/constraints lab8/lef lab8/physical_design lab8/netlist
cd lab8
```

2. Copy the gate-level netlist from Experiment 6 into the netlist/ sub-directory:

```
cp ../lab7/synthesis/Adder_32bit_8_stage_netlist.v netlist/
```

3. Copy the SDC constraint file from Experiment 6 into the constraints/ sub-directory:

```
cp ../lab7/synthesis/Adder_32bit_8_stage.sdc constraints/
```

4. Copy the .lib timing library files from Experiment 6 into the lib/ sub-directory:

```
cp ../lab7/lib/*.lib lib/
```

5. Copy the LEF (Library Exchange Format) file, which describes the physical geometry of the standard cells and routing layers, into the lef/ sub-directory. Ask your lab instructor for the correct path on your machine if the one below does not exist:

```
cp ~/sahmed/eee442/lab4/lef/all.lef lef/
```

6. Go to physical_design/ and invoke the Innovus tool to place and route the design. This launches Innovus in GUI mode:

```
cd physical_design  
innovus
```

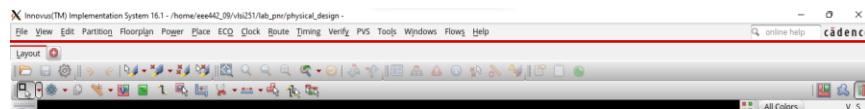


Figure 1: Innovus main window and toolbar after launching the tool.

Part 2: Design Import

7. Click on File → Import Design. This opens the Design Import window.
8. Under Netlist, choose Verilog. Click the browse button and select the gate-level netlist from the netlist/ sub-directory. Make sure your netlist is correct.
9. Under Top Cell, select Auto Assign.
10. Under Technology/Physical Libraries, choose LEF Files. Click the browse button and select the all.lef file from the lef/ sub-directory.
11. Under Power, set VDD as the Power Nets and VSS as the Ground Nets. Then click OK.

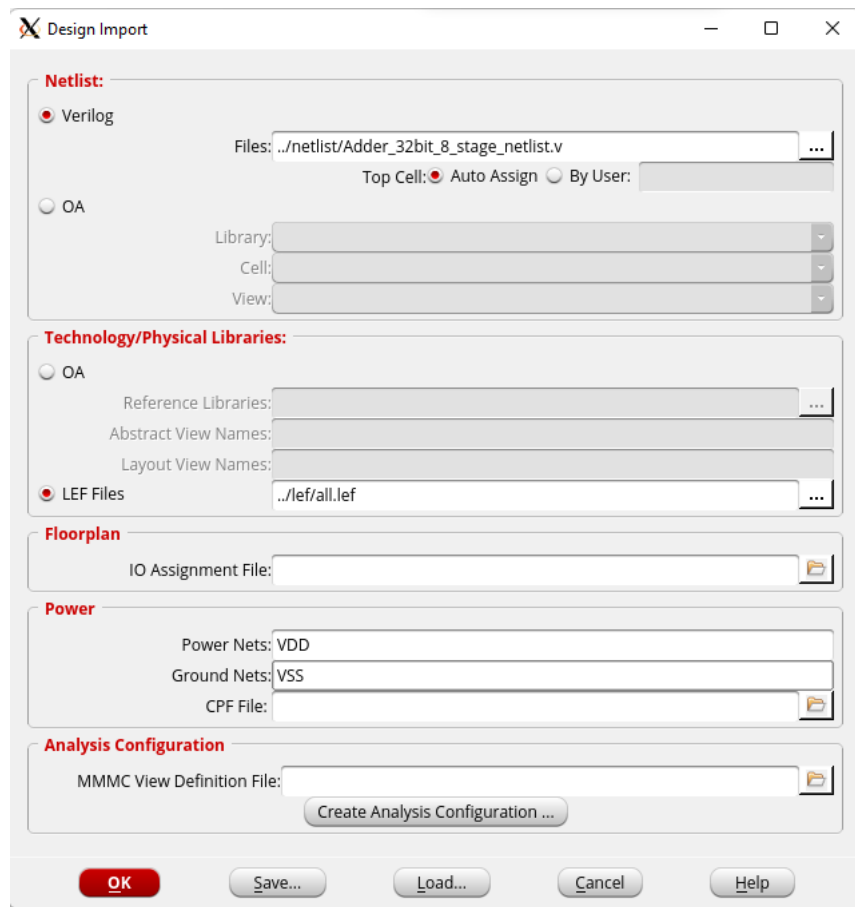


Figure 2: Design Import window, with the Verilog netlist, LEF file, and VDD/VSS power nets configured.

12. Under Analysis Configuration, click the Create Analysis Configuration button. This opens the MMMC (Multi-Mode Multi-Corner) Browser window.

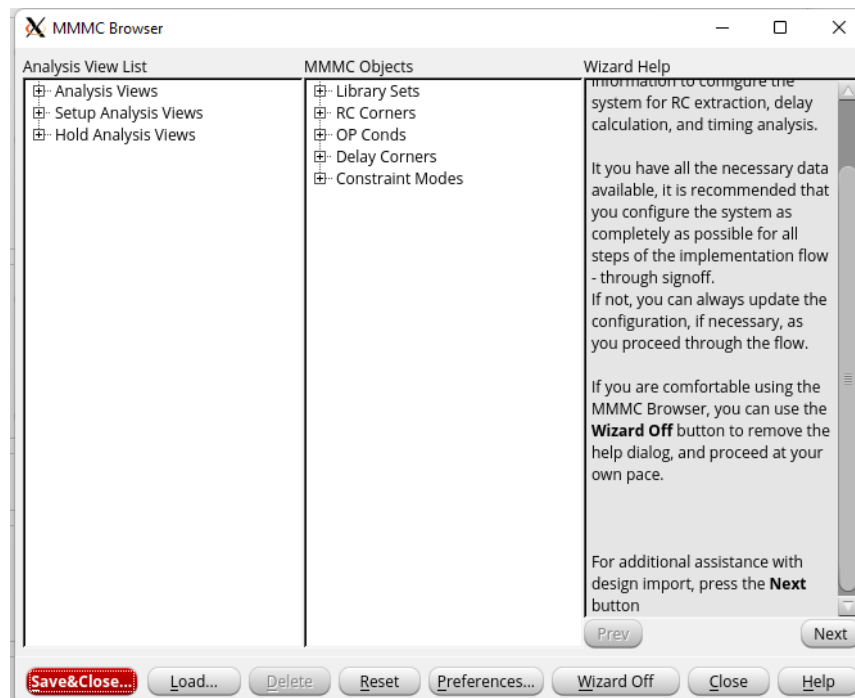


Figure 3: MMMC Browser window, used to configure library sets, RC corners, and constraint modes for timing analysis.

13. Double-click Library Sets. In the Add Library Set window, name it max_timing and click Add under Timing Library Files to attach your .lib file(s).

Note: Steps 15 to 27 of the original walkthrough configure the remaining MMMC objects (RC corners, OP conditions, delay corners, setup/hold analysis views, and constraint modes) in essentially the same way as the Library Set step above: double-click the object type on the left, name it, and attach the relevant file(s). These are normally demonstrated live by the instructor; use the Wizard Help panel on the right of the MMMC Browser (Figure 3) for guidance on each one, and use Save & Close once all objects are configured.

14. Once the analysis configuration is complete, click OK to finish Design Import. The floorplan view opens, initially with the design imported but no floorplan defined yet.

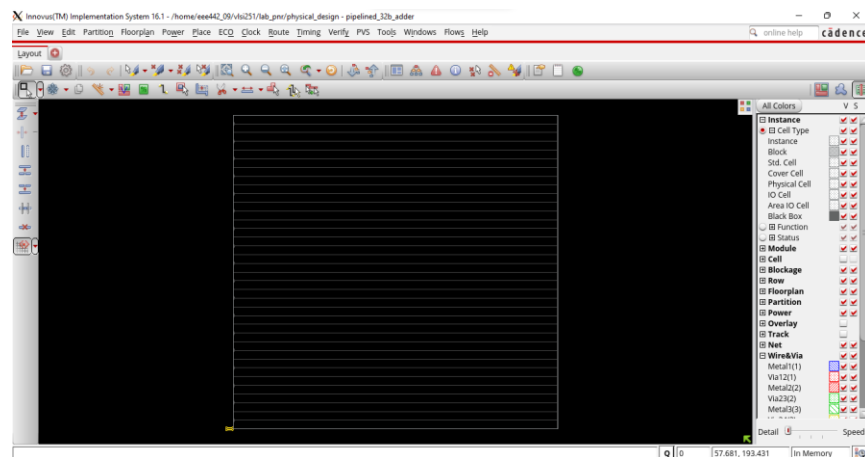


Figure 4: Innovus layout window immediately after Design Import, showing the imported netlist (pipelined_32b_adder / Adder_32bit_8_stage) with no floorplan yet.

Part 3: Floor Planning

15. Click Floorplan → Specify Floorplan. This opens the Specify Floorplan window.

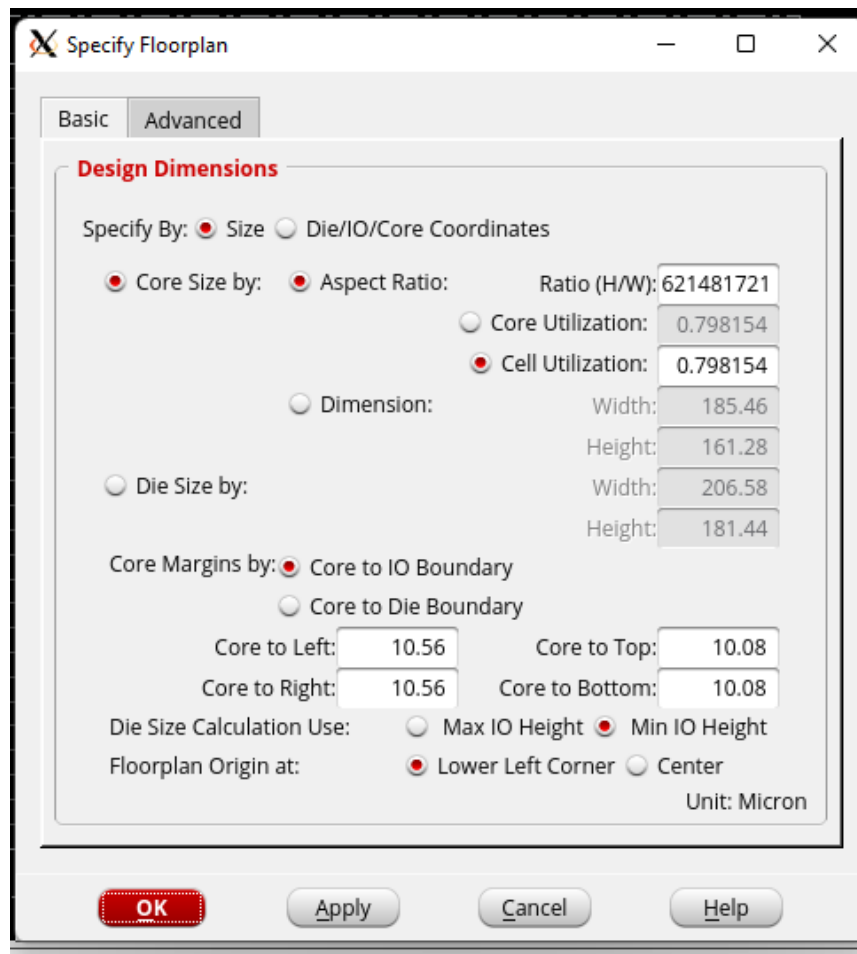


Figure 5: Specify Floorplan window.

16. Select an aspect ratio as required. Give suitable dimensions for Core to Left, Core to Right, Core to Top, and Core to Bottom (remember these values are in microns) to leave space for the power rings. After defining the core area, click OK.

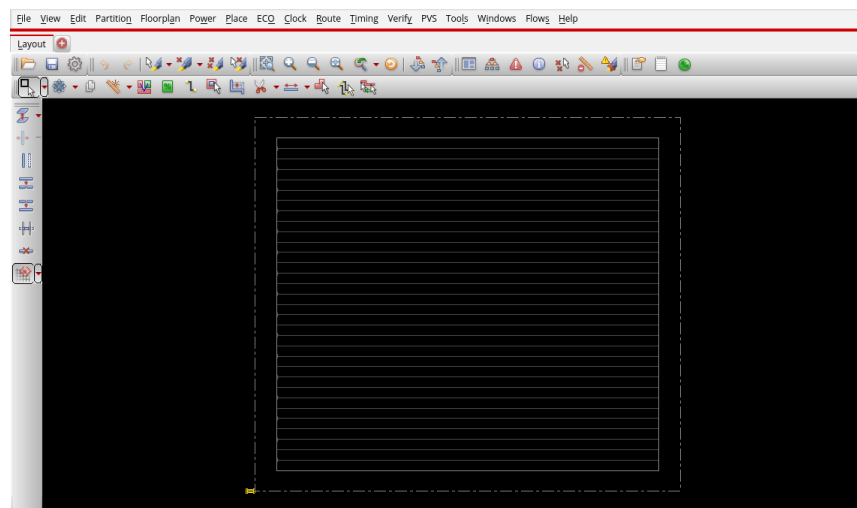


Figure 6: Floorplan generated from the Specify Floorplan settings, showing the die/IO boundary (dashed) and the core boundary (solid).

Note: Save the design separately at every stage of the flow. If an error occurs later, you can restore the most recent

save and continue from there instead of starting over - this saves a great deal of time.

17. Save the current design. Click File → Save Design, give it a descriptive name (e.g., pipelined_32b_adder_floor_plan), and click OK.

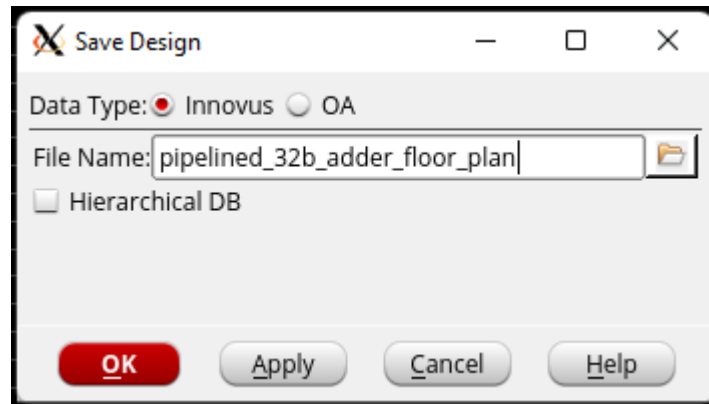


Figure 7: Save Design window.

18. To reload a saved stage later, click File → Restore Design and select the saved design file.

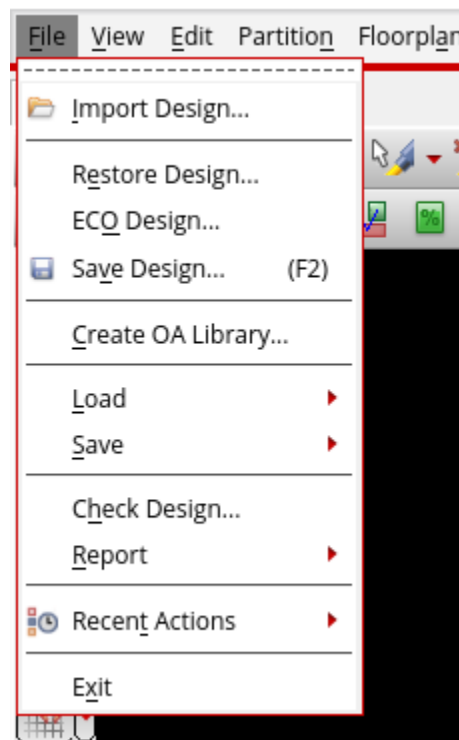


Figure 8: File menu, showing the Restore Design option.

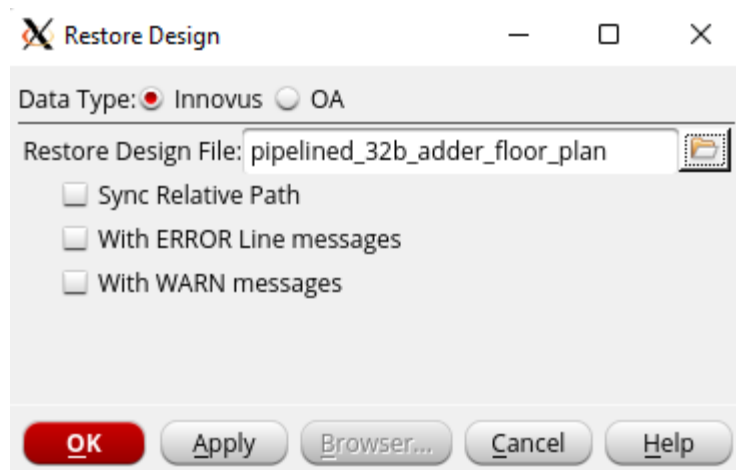


Figure 9: Restore Design window.

Part 4: Power Planning

Add Rings

19. Click Power → Power Planning → Add Rings. This opens the Add Rings window.

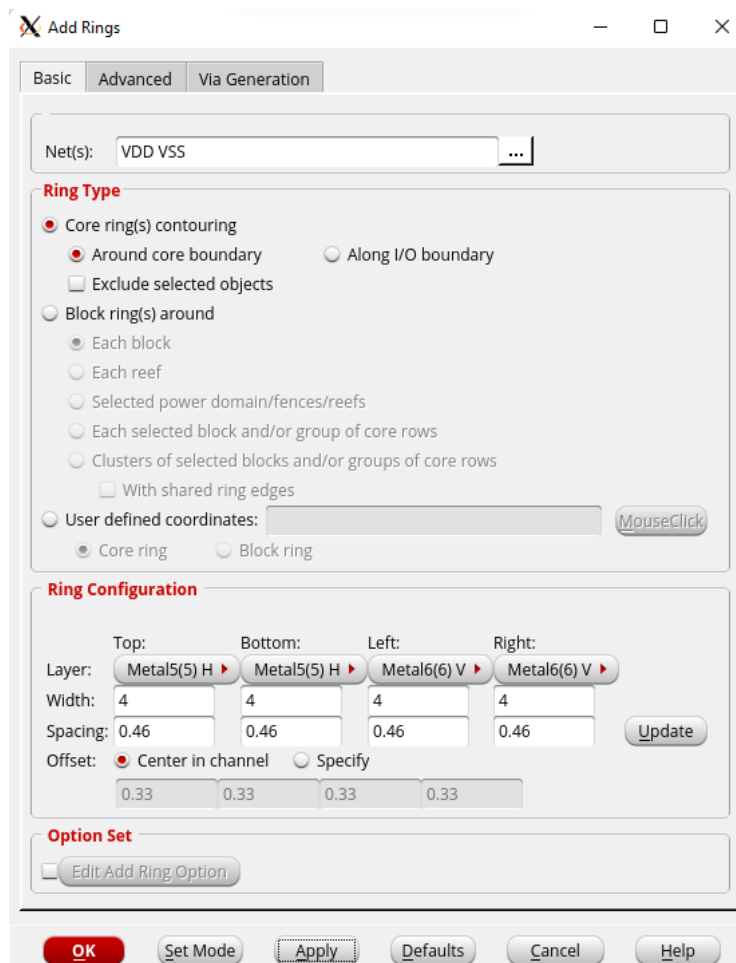


Figure 10: Add Rings window.

20. Type or select VDD and VSS under Net(s).

21. Select the top/bottom layer as Metal5 and the left/right layer as Metal6. Make sure these metal layer names match those in the all.lef file in your lef/ sub-directory. Set the width/spacing as required (units in microns), leaving space between the core boundary and the I/O pads. Select Center in channel for the offset and click OK.

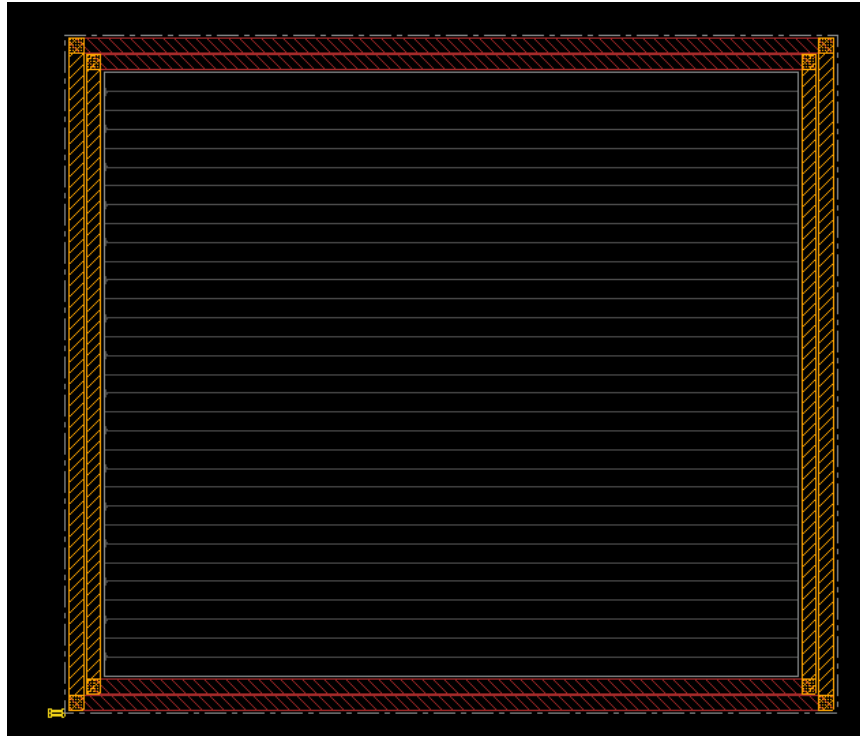


Figure 11: Power ring generated between the core boundary and the I/O/die boundary.

Add Stripes

22. Click Power → Power Planning → Add Stripe. This opens the Add Stripes window.
23. Type or select VDD and VSS under Net(s).
24. Select Layer as Metal5 and Direction as Horizontal. Make sure the metal layer name and direction match those in all.lef. Set the width/spacing as required. Choose a Set Pattern option, use the core ring as the Stripe Boundary, and set the first/last stripe starting position. Click OK.



Figure 12: Add Stripes window, configured for a horizontal Metal5 stripe.

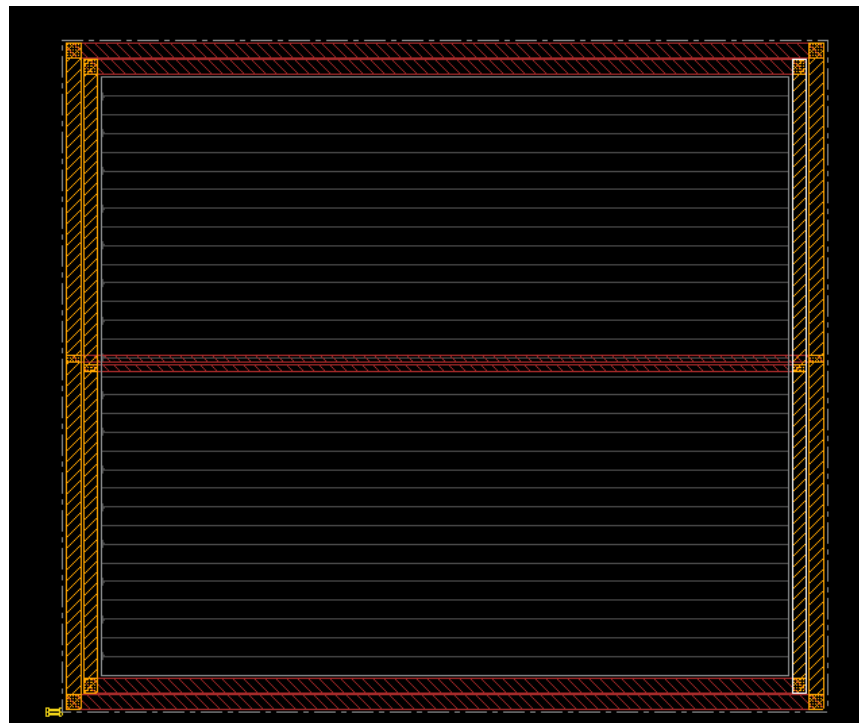


Figure 13: Horizontal power stripe added across the core.

25. Repeat Add Stripe for the vertical direction: select Layer as Metal6 and Direction as Vertical, with the stripe boundary set to the core ring, and click OK.

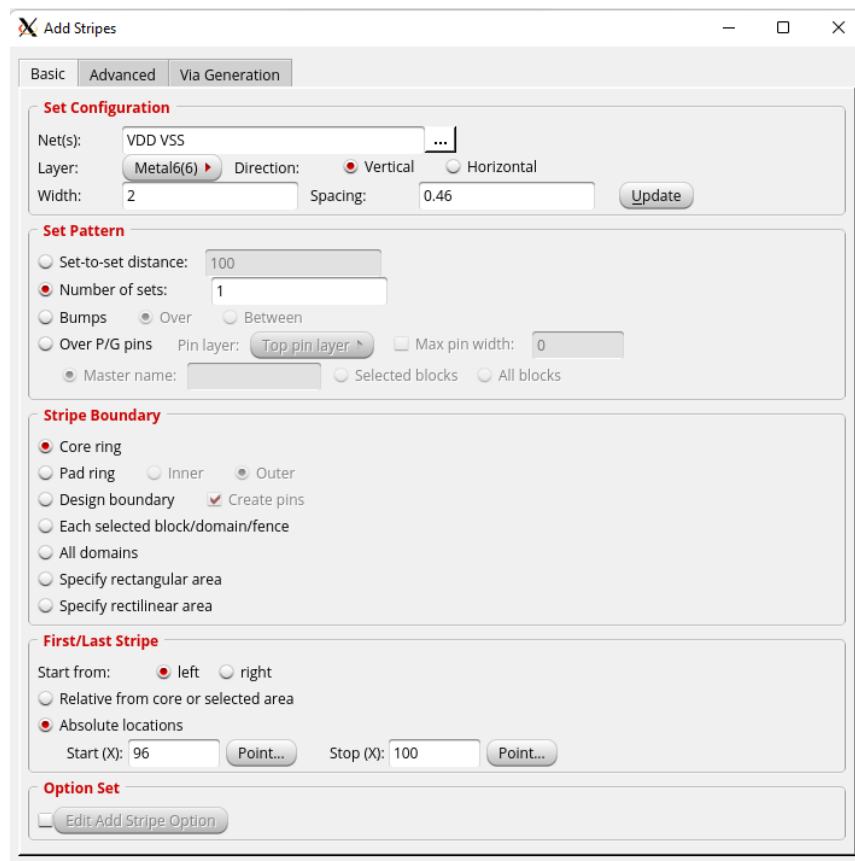


Figure 14: Add Stripes window, configured for a vertical Metal6 stripe.

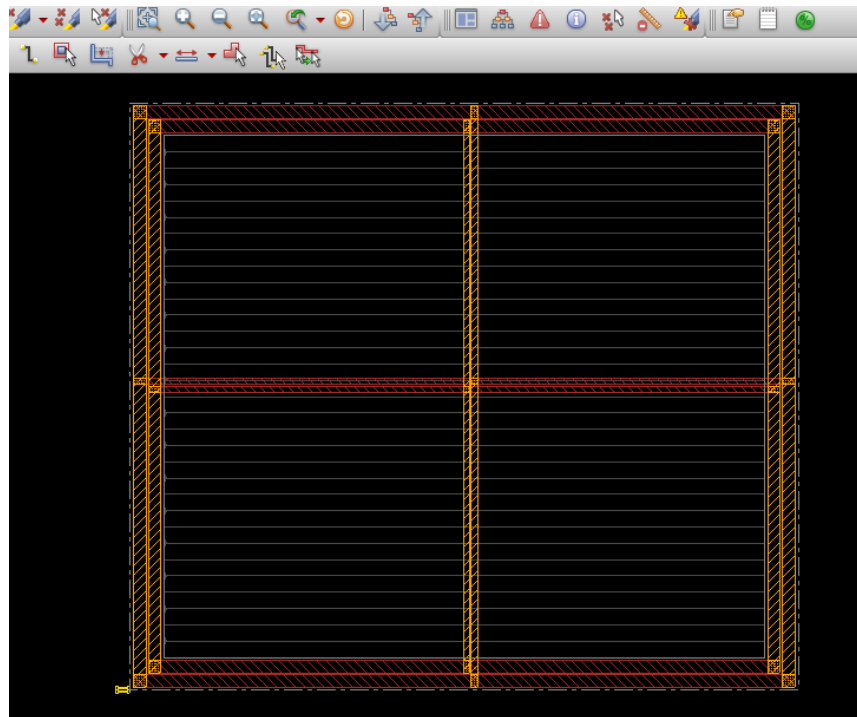


Figure 15: Vertical power stripe added, completing the power grid together with the horizontal stripe.

Special Route (Standard Cell Rails)

26. Click Route → Special Route. This opens the SRoute window.

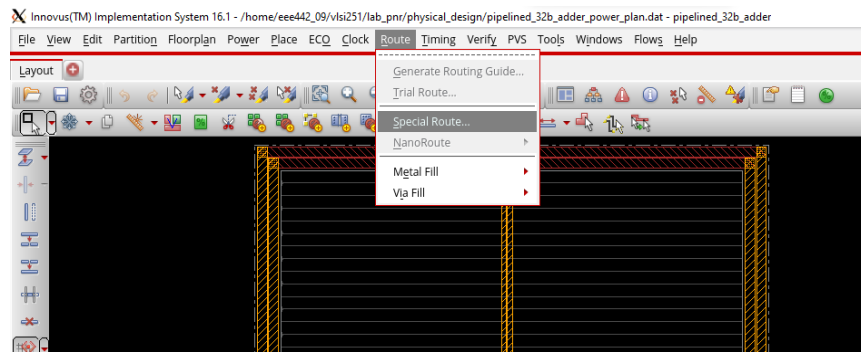


Figure 16: Route → Special Route menu.

27. Type or select VDD and VSS under Net(s). Choose Follow Pins (for standard cells) under SRoute, along with the top/bottom layer and any other options needed. Click OK.

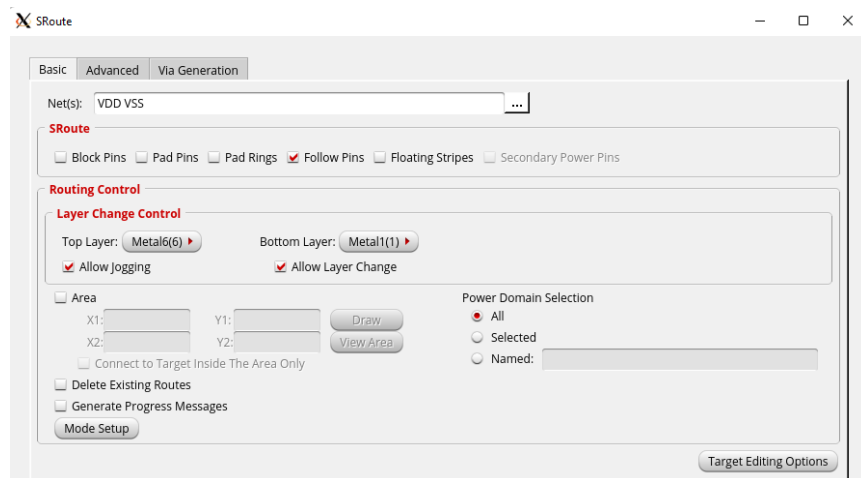


Figure 17: SRoute window, configured to follow pins with Metal6 as the top layer and Metal1 as the bottom layer.

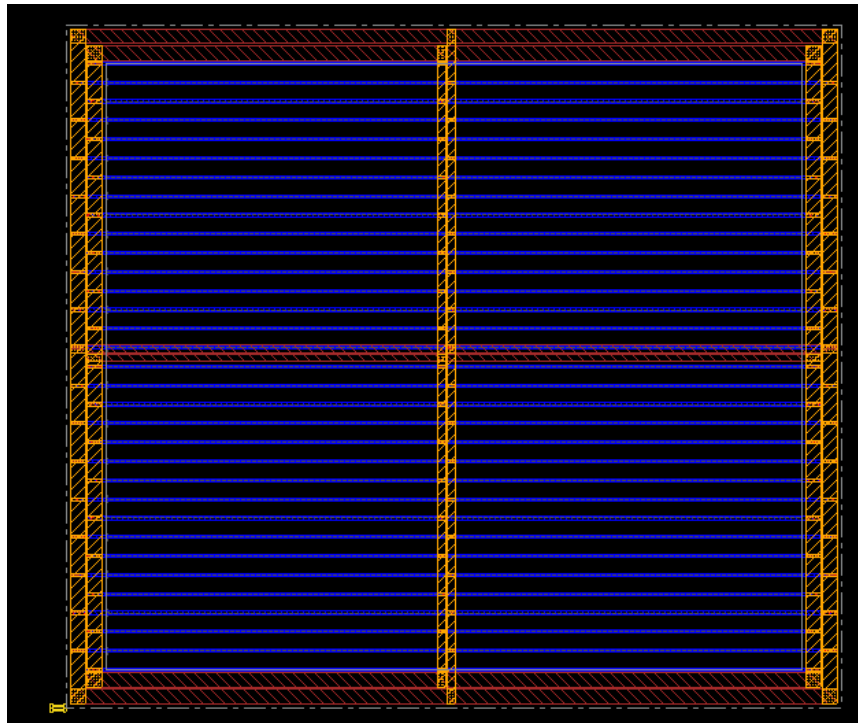


Figure 18: Standard cell power rails created by SRoute, connecting every cell row to the VDD/VSS grid.

Part 5: Pin Placement

28. Click Edit → Pin Editor. This opens the Pin Editor window.

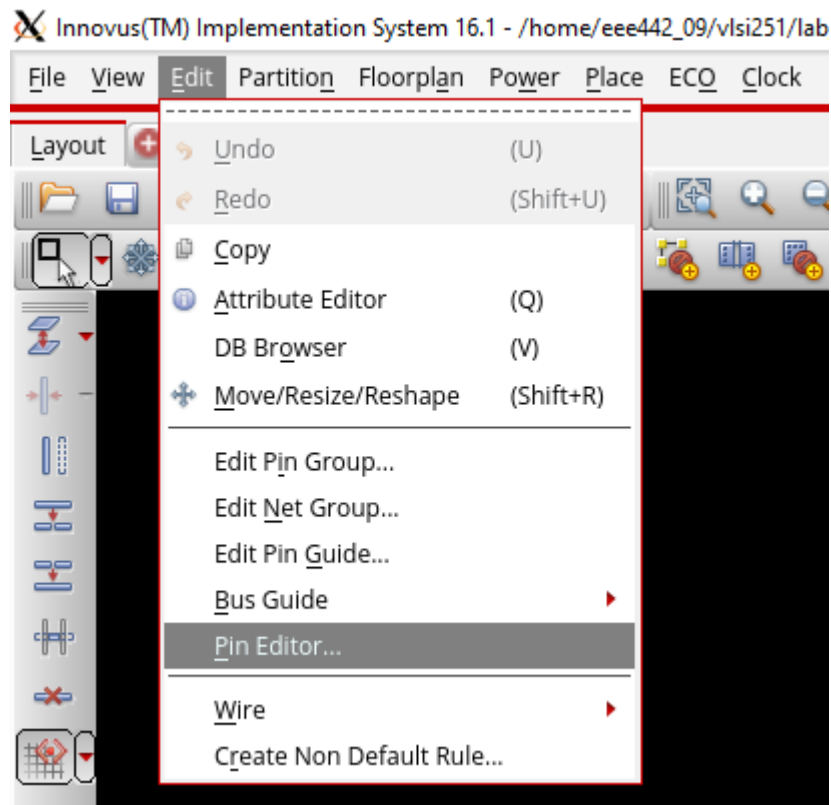


Figure 19: Edit → Pin Editor menu.

29. In the Pin Group list, select a pin (for example, clk). Under Pin Attribute, choose the Side/Edge (e.g., Left) and Layer (e.g., Metal3) for that pin.

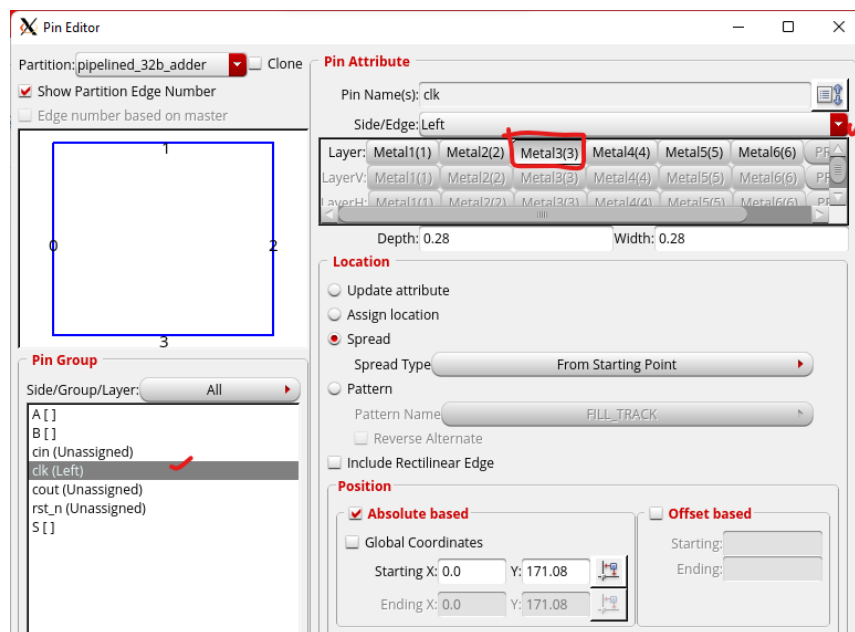


Figure 20: Pin Editor window, assigning the clk pin to the Left edge on Metal3.

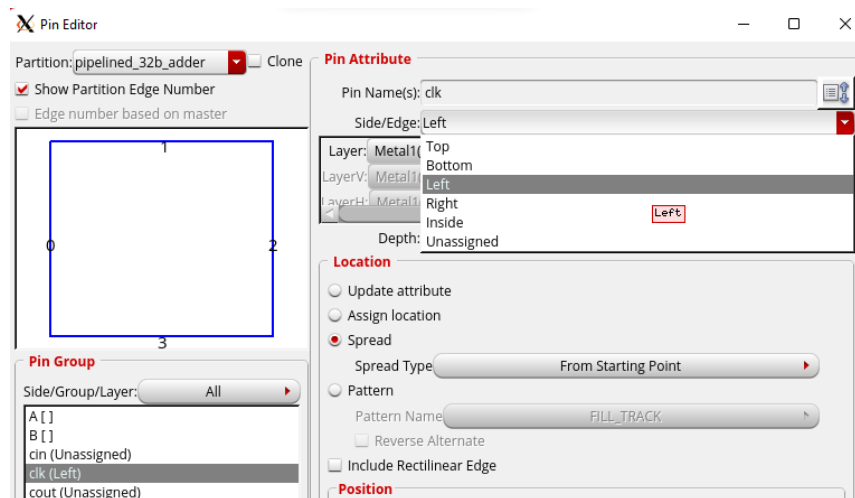


Figure 21: Pin Editor Side/Edge drop-down, choosing which edge of the die a pin group is placed on.

30. Under Location, choose Spread with a Spread Type (e.g., From Starting Point), set the starting position and spacing between pins, and choose a direction (e.g., Clockwise). Repeat this for each remaining pin group (A, B, cin, cout, rst_n, S) one at a time, choosing the location where you want to place each group of pins.

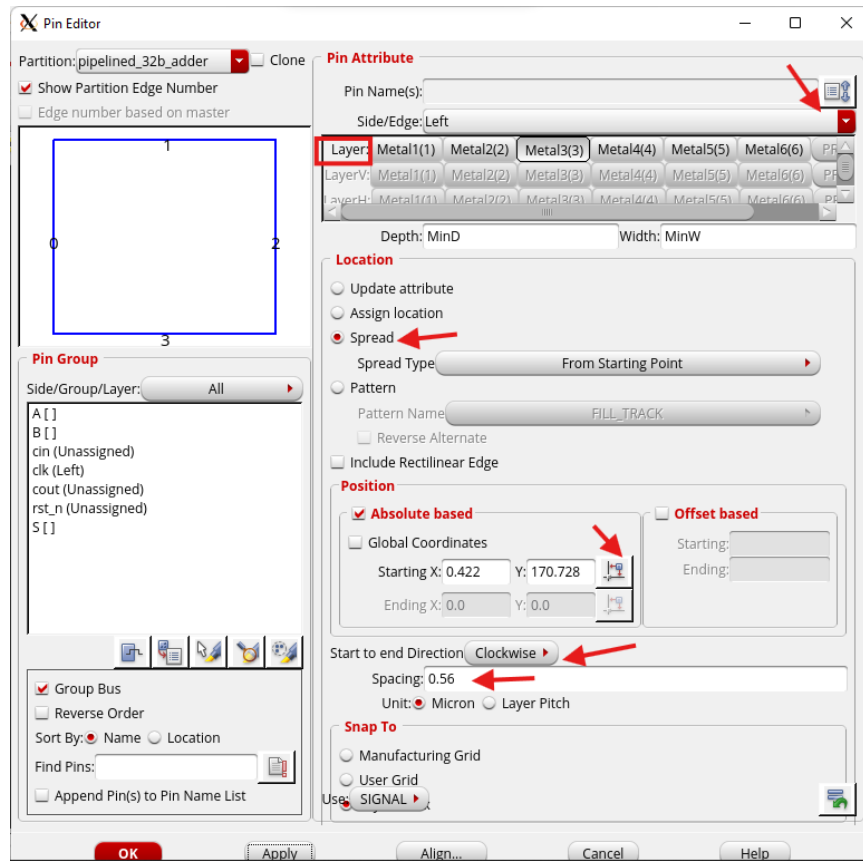


Figure 22: Pin Editor with the spread direction, starting position, and pin-to-pin spacing highlighted.

31. After completing pin placement, all pins should appear around the edges of the core, ready for routing.

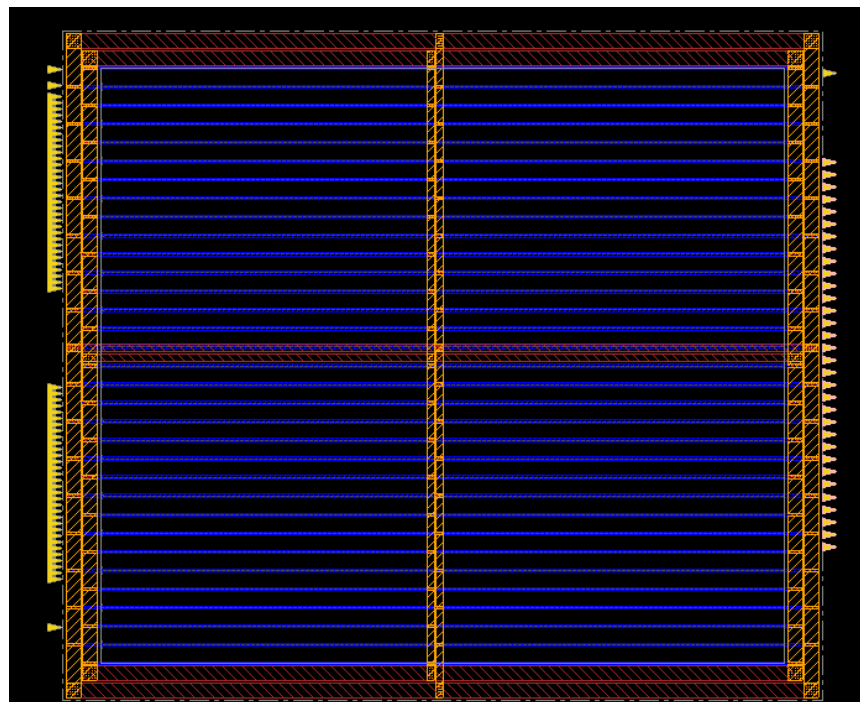


Figure 23: Completed pin placement, with pins spread along the left and right edges of the design.

Part 6: Cell Placement

32. Click Place → Place Standard Cells. This opens the Place window.

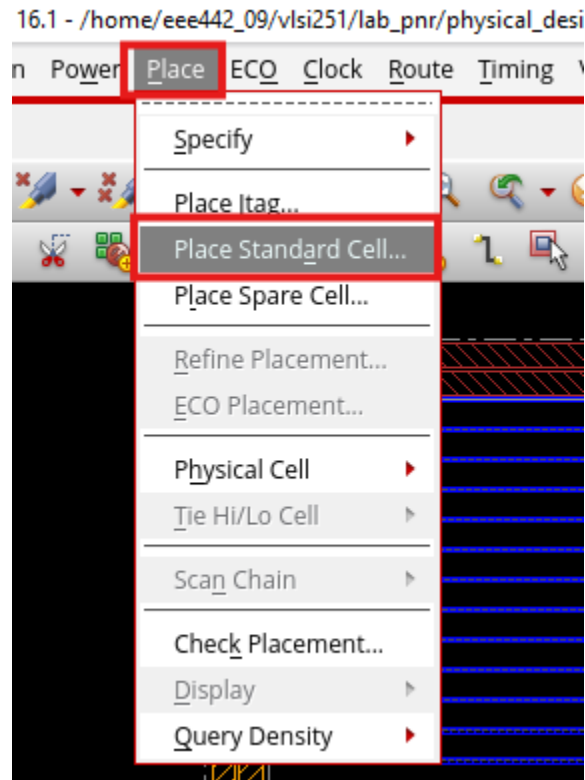


Figure 24: Place → Place Standard Cells menu.

33. Select Run Full Placement, and choose Include Pre-Place Optimization under Optimization Options. Click OK.

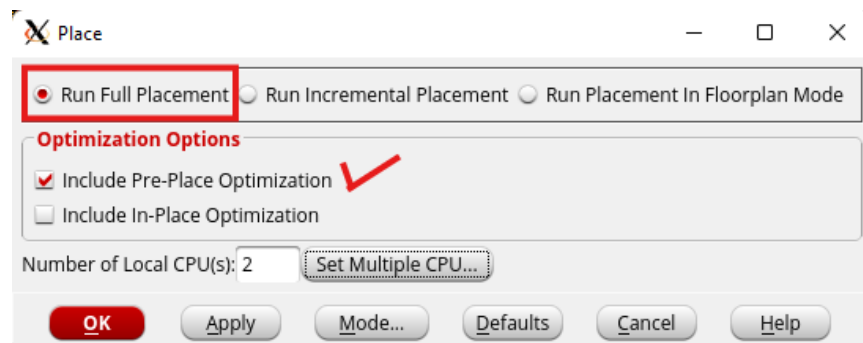


Figure 25: Place window, configured for a full placement run with pre-place optimization.

34. Click the Physical View button in the toolbar to see the placed standard cells and their routing.

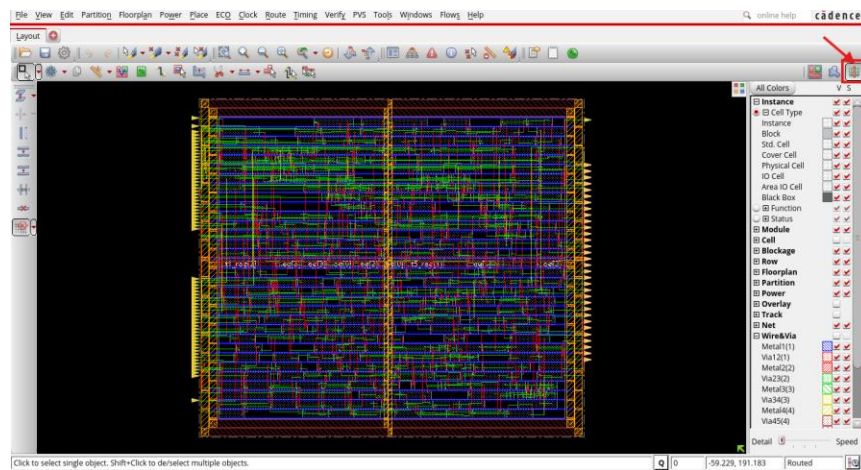


Figure 26: Physical view after placement, showing the standard cells and their interconnect.

Part 7: Pre-CTS Timing and Optimization

35. Click Timing → Report Timing. This opens the Timing Analysis window.

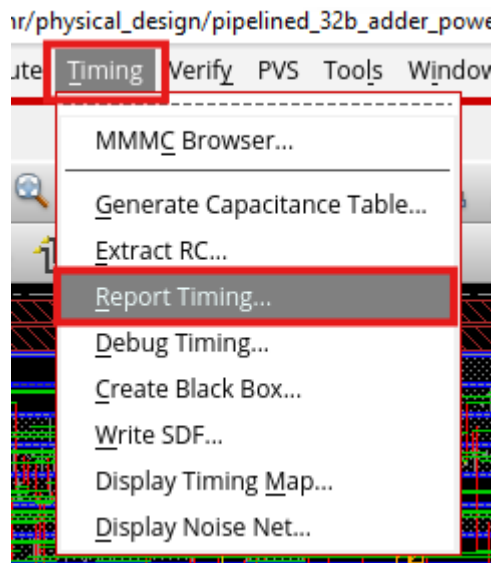


Figure 27: Timing → Report Timing menu.

36. Select Pre-CTS under Design Stage and Setup under Analysis Type. Click OK. This displays the timing information in the Innovus terminal; carefully check the Worst Negative Slack (WNS) and Total Negative Slack (TNS).

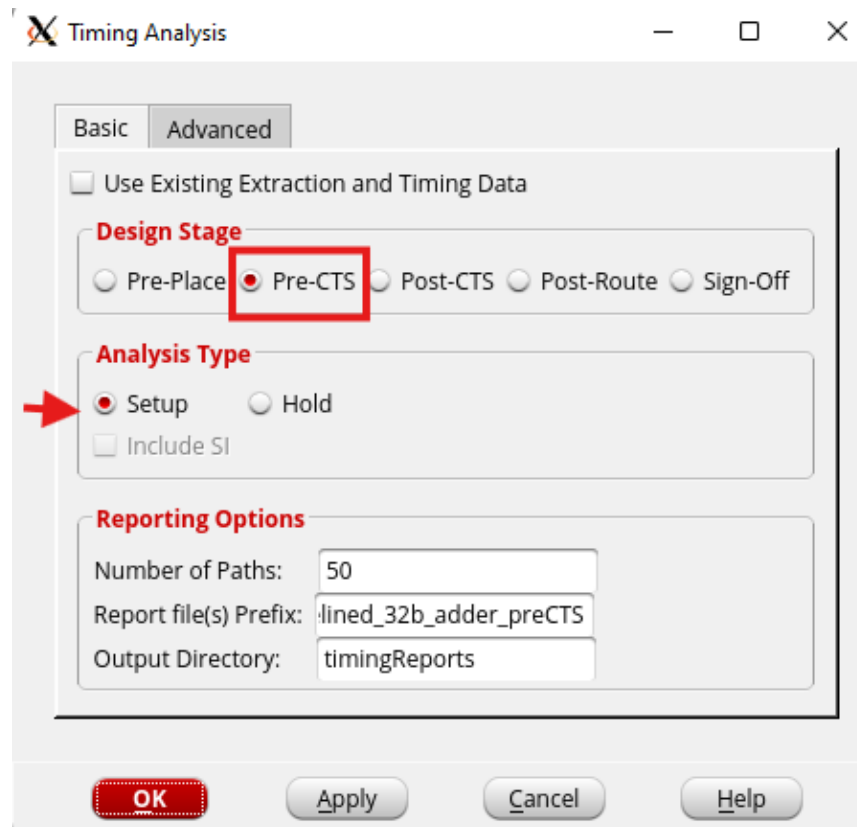


Figure 28: Timing Analysis window, configured for a Pre-CTS setup report.

timeDesign Summary				
Setup views included: worst_case				
Setup mode	all	reg2reg	default	
WNS (ns):	-0.890	-0.274	-0.890	
TNS (ns):	-5.958	-3.434	-2.524	
Violating Paths:	36	32	4	
All Paths:	528	203	325	

DRVs	Real		Total
	Nr nets(terms)	Worst Vio	Nr nets(terms)
max_cap	0 (0)	0.000	0 (0)
max_tran	0 (0)	0.000	0 (0)
max_fanout	0 (0)	0	0 (0)
max_length	0 (0)	0	0 (0)

Density: 79.548%
Routing Overflow: 0.00% H and 0.00% V

Reported timing to dir timingReports
Total CPU time: 0.9 sec
Total Real time: 2.0 sec
Total Memory Usage: 1413,324219 Mbytes
innovus 1>

Figure 29: timeDesign Summary showing a negative WNS (-0.890 ns) and TNS (-5.958 ns) before optimization, with 36 violating paths.

37. If there is a negative WNS/TNS, click ECO → Optimize Design. This opens the Optimization window.

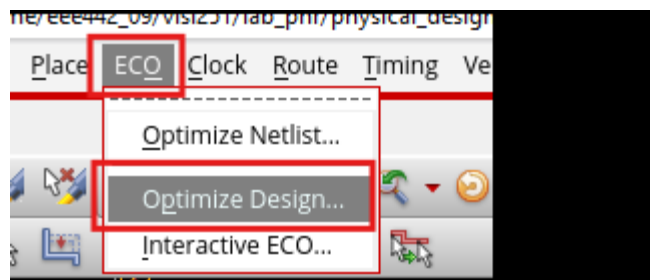


Figure 30: ECO → Optimize Design menu.

38. Select Pre-CTS under Design Stage and Setup, Max Cap, and Max Tran under Optimization Type. Click OK. This optimizes the design; check timing again, and repeat until timing is met.

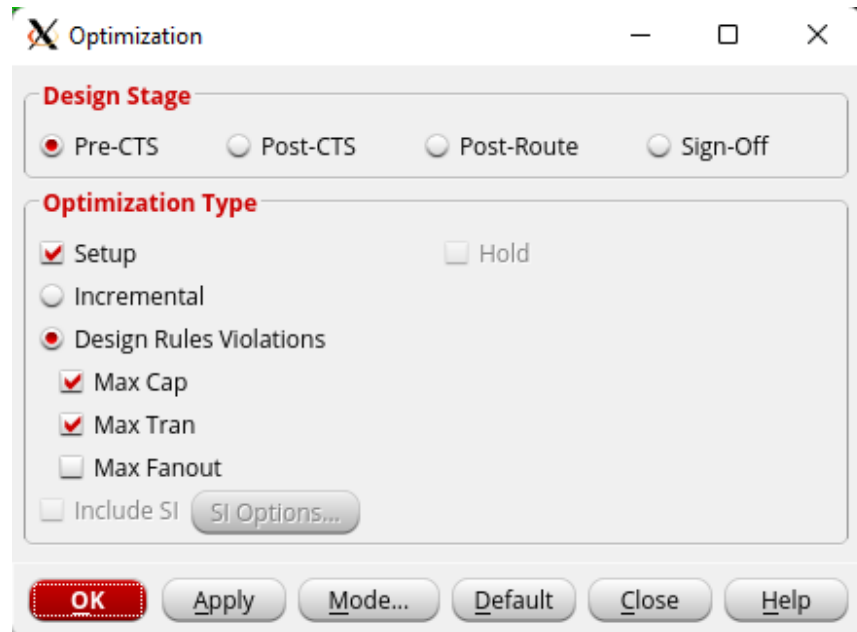


Figure 31: Optimization window, configured for Pre-CTS setup optimization with Max Cap and Max Tran design-rule fixing.

timeDesign Summary			
Setup views included: worst_case			
Setup mode	all	reg2reg	default
WNS (ns):	0.002	0.002	0.009
TNS (ns):	0.000	0.000	0.000
Violating Paths:	0	0	0
All Paths:	528	203	325

DRVs	Real		Total
	Nr nets(terms)	Worst Vio	Nr nets(terms)
max_cap	0 (0)	0.000	0 (0)
max_tran	0 (0)	0.000	0 (0)
max_fanout	0 (0)	0	0 (0)
max_length	0 (0)	0	0 (0)

Density: 81.294%
 Routing Overflow: 0.00% H and 0.00% V

Reported timing to dir timingReports
 Total CPU time: 0.83 sec
 Total Real time: 2.0 sec
 Total Memory Usage: 1565.941406 Mbytes

innovus 1>

Figure 32: timeDesign Summary after optimization, showing WNS and TNS both at or above zero (0.002 ns / 0.000 ns) with no remaining violating paths.

In practice, a design may need several rounds of optimization - the design used for these reference screenshots needed about six rounds - before the WNS and TNS both become non-negative. Keep re-running report timing and ECO → Optimize Design until they do.

Part 8: Clock Tree Synthesis (CTS)

39. In your physical_design/ folder, use an editor (gedit) to create the CTS script:

gedit CTS.tcl &

Paste the following contents and save it:

```
# Sourcing variables
#source setup.tcl
set MAX_CLOCK_ROUTING_LAYER "Metal4"
set MIN_CLOCK_ROUTING_LAYER "Metal3"
set step cts_opt

# Load previous Innovus database
if { [dbGet top.name] eq 0x0 } {
    source FLOW_DBS/placeopt.enc
}

set_interactive_constraint_modes [all_constraint_modes -active]

## Reload lib setting
setDontUse CLKINV* false
set_dont_touch [get_lib_cells "*/CLKINV*"] false

# Settings CCOPT

create_ccopt_clock_tree_spec -file ./scripts/clock1.spec
source ./scripts/clock1.spec
```

```

# CCOPT Route type

set CLOCK_ROUTING_LAYER_RANGE ""
if {[info exists MAX_CLOCK_ROUTING_LAYER] && $MAX_CLOCK_ROUTING_LAYER!=""} {
    append CLOCK_ROUTING_LAYER_RANGE " -top_preferred_layer $MAX_CLOCK_ROUTING_LAYER"
}
if {[info exists MIN_CLOCK_ROUTING_LAYER] && $MIN_CLOCK_ROUTING_LAYER!=""} {
    append CLOCK_ROUTING_LAYER_RANGE " -bottom_preferred_layer $MIN_CLOCK_ROUTING_LAYER"
}

create_route_type -name CLOCK_TRUNK -non_default_rule CLOCK_CLK_NONDEFAULT_ROUTE_RULE \
    -shield_net VSS -top_preferred_layer $MAX_CLOCK_ROUTING_LAYER \
    -bottom_preferred_layer $MIN_CLOCK_ROUTING_LAYER

set clock_list [get_ccopt_clock_trees *]
foreach clock $clock_list {
    set ccopt_property route_type -net_type top -clock_tree $clock CLOCK_TRUNK
    set_ccopt_property route_type -net_type trunk -clock_tree $clock CLOCK_TRUNK
}

# CCOPT Specify clock cells
set_ccopt_property use_inverters true

# CCOPT Target MAX TRANS, MAX SLEW, MAX FANOUT
if {[info exists LIMIT_SLEW(CLOCK)] && $LIMIT_SLEW(CLOCK)!=""} {
    set_ccopt_property target_max_trans $LIMIT_SLEW(CLOCK)
}
if {[info exists CLOCK_MAX_SKEW] && $CLOCK_MAX_SKEW!=""} {
    set_ccopt_property target_skew $CLOCK_MAX_SKEW
}

if {[info exists LIMIT_FANOUT] && $LIMIT_FANOUT!=""} {
    set_ccopt_property max_fanout $LIMIT_FANOUT
}

setTrialRouteMode -highEffort true -handlePreroute true -useM1 true \
    -maxRouteLayer $MAX_SIGNAL_ROUTING_LAYER
setAnalysisMode -asyncChecks async
setAnalysisMode -cpr both -analysisType onChipVariation
setCTSMode -rcCorrelationAutoMode false
set_global timing_cpr_threshold_ps 5
setOptMode -reclaimArea default

# CCOPT Main
ccopt_design -cts
set ::CTE::timing_save_source_latency_per_view 1

set_clock_latency -source -early -min 0.00 [get_ports clk] -clock clk
set_clock_latency -source -early -max 0.00 [get_ports clk] -clock clk
set_clock_latency -source -late -min 0.00 [get_ports clk] -clock clk
set_clock_latency -source -late -max 0.00 [get_ports clk] -clock clk

# Resetting the dont_use list
if {[info exists CLOCK_TREE_INV_MODEL] && $CLOCK_TREE_INV_MODEL!=""} {
    foreach clock_model $CLOCK_TREE_INV_MODEL { setDontUse $clock_model true }
}
if {[info exists CLOCK_TREE_BUF_MODEL] && $CLOCK_TREE_BUF_MODEL!=""} {
    foreach clock_model $CLOCK_TREE_BUF_MODEL { setDontUse $clock_model true }
}

# Apply timing derate
set_timing_derate 1.02 -late -clock -delay_corner WC_rcworst
set_timing_derate 1.02 -late -data -delay_corner WC_rcworst
set_timing_derate 0.98 -early -clock -delay_corner WC_rcworst
set_timing_derate 1.0 -early -clock -delay_corner BC_rcbest

```

```

set_timing_derate 1.0 -early -data -delay_corner BC_rcbest
set_timing_derate 1.14 -late -clock -delay_corner BC_rcbest

setDelayCalMode -SIAware false
set_global timing_remove_clock_reconvergence_pessimism true
set_global timing_cpvr_threshold_ps 10
set_global timing_cpvr_transition_sense normal

set_global timing_enable_timing_window_pessimism_removal true
setExtractRCMode -total_c_th 0.0
setSIMode -num_si_iteration 4
setSIMode -enable_logical_correlation_in_delay true
setSIMode -individual_attacker_simulation_filtering true
setSIMode -individual_attacker_threshold 0.01
setSIMode -nonlinear_attacker_slew false
setSIMode -receiver_clk_peak_limit 0.1

setOptMode -powerEffort high -leakageToDynamicRatio 1.0
setOptMode -setupTargetSlack 0
setOptMode -effort high
set_interactive_constraint_modes [all_constraint_modes -active]
set_false_path -hold -from [all_inputs]
set_false_path -hold -to [all_outputs]
setMaxRouteLayer $MAX SIGNAL ROUTING LAYER
setOptMode -reclaimArea default

# Main Optimization
optDesign -postCTS

# HOLD Optimization
if {[info exists COARSE_HOLD_BUFFER_REGEX] && $COARSE_HOLD_BUFFER_REGEX!=""} {
    foreach hold_model $COARSE_HOLD_BUFFER_REGEX { setDontUse $hold_model false }
    setOptMode -holdFixingCells [dbGet head.allCells.name $COARSE_HOLD_BUFFER_REGEX]
}

setAnalysisMode -checkType hold
optDesign -postCTS -hold

puts "TimeDesign Final"
timeDesign -postCTS

## Reload lib setting
setDontUse CLKINV* true
set_interactive_constraint_modes [all_constraint_modes -active]
set_dont_touch [get_lib_cells "*/CLKINV*"] true

# Save database
set enc_save_portable_design 1
saveDesign FLOW DBS/$step.enc -compress

```

Listing 1: CTS.tcl - configures clock-tree routing and cell settings, builds the clock tree (ccopt_design -cts), applies timing derates, runs post-CTS setup and hold optimization, and saves the design.

Note: This script has been cleaned up from the version handed out in the lab sheet: a few lines that had wrapped across the printed page have been rejoined into single Tcl commands, and the \$ prefix has been restored before variable references (for example, \$MAX_CLOCK_ROUTING_LAYER, \$step) where it had been dropped. Variables such as MAX_SIGNAL_ROUTING_LAYER, LIMIT_SLEW, CLOCK_MAX_SKEW, LIMIT_FANOUT, COARSE_HOLD_BUFFER_REGEX, CLOCK_TREE_INV_MODEL, and CLOCK_TREE_BUF_MODEL are expected to already be defined by a setup.tcl sourced earlier in your flow, as the commented-out #source setup.tcl line at the top suggests; if you have not sourced such a file, define these variables yourself, or the corresponding if-blocks will simply be skipped.

40. Source the CTS script from the Innovus command line:

source CTS.tcl

```

innovus 1> source setup.tcl
**WARN: (IMPOPT-3058): Cell tsmc18/CLKINVL already has a dont_use attribute false.
Type 'man IMPOPT-3058' for more detail.
Creating clock tree spec for modes (timing configs): Adder_32_constraints
extract_clock_generator_skew_groups=true; create_ccopt_clock_tree_spec will generate skew
groups with a name prefix of "_clock_gen" to balance clock generator connected flops with
the clock generator they drive.
Analyzing clock structure...
Analyzing clock structure done.
Wrote: ./scripts/clock1.spec
Extracting original clock gating for clk...
clock_tree clk contains 264 sinks and 0 clock gates.
Extraction for clk complete.
Extracting original clock gating for clk done.

```

Figure 33: Terminal output after sourcing CTS.tcl, showing the clock tree specification being generated and the clock structure analyzed.

41. Once CTS completes, view the generated clock tree in the layout: click on the clk pin to highlight the clock net and its buffers/sinks.

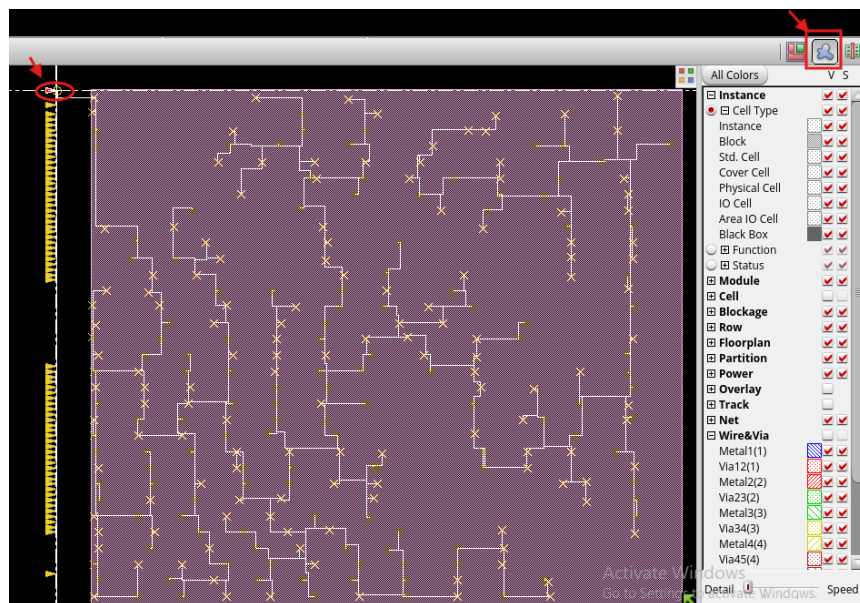


Figure 34: Clock tree structure after CTS, showing the buffered clock network (yellow) fanning out to its sink flip-flops (white crosses).

Part 9: Post-CTS Timing Report

42. Report timing again, this time selecting Post-CTS under Design Stage, to confirm that timing is still met once the clock tree and its associated delay and skew are taken into account.

timeDesign Summary			
Setup views included: worst_case			
Setup mode	all	reg2reg	default
WNS (ns):	0.002	0.002	0.009
TNS (ns):	0.000	0.000	0.000
Violating Paths:	0	0	0
All Paths:	528	203	325
DRVs	Real		Total
	Nr nets(terms)	Worst Vio	Nr nets(terms)
max_cap	0 (0)	0.000	0 (0)
max_tran	0 (0)	0.000	0 (0)
max_fanout	0 (0)	0	0 (0)
max_length	0 (0)	0	0 (0)
Density: 81.294%			
Routing Overflow: 0.00% H and 0.00% V			
Reported timing to dir timingReports			

Figure 35: Post-CTS timeDesign Summary, confirming WNS and TNS remain non-negative after clock tree synthesis.

Part 10: Geometry and Connectivity Verification

Before routing (or after a first-pass route), the design should be checked for physical rule violations and connectivity problems.

43. Click Verify → Verify Geometry. This opens the Verify Geometry window.

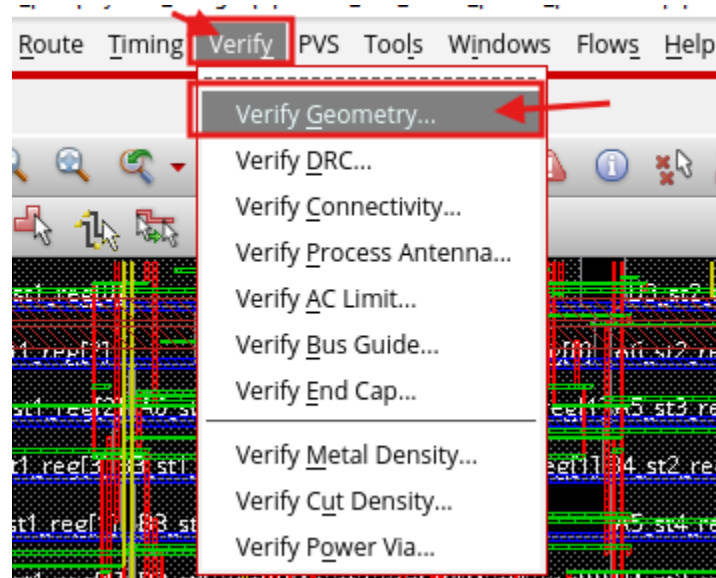


Figure 36: Verify → Verify Geometry menu.

44. Leave Entire area selected, keep the default set of Minimum Width / Spacing / Area, Short, Cell Overlap, Via Enclosure, and related checks enabled, and click OK.



Figure 37: Verify Geometry window, with the full set of design-rule checks enabled.

45. White crosses in the layout mark the reported errors; these must be resolved.

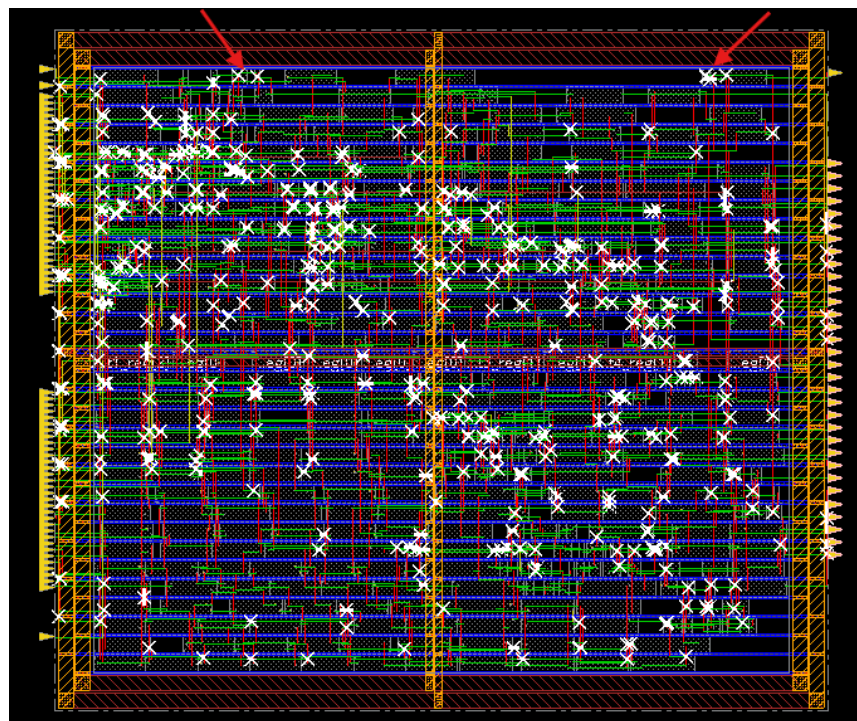


Figure 38: Geometry violations shown as white crosses scattered across the design.

46. Click Verify → Verify Connectivity to check for opens, shorts, and other net-connectivity problems.

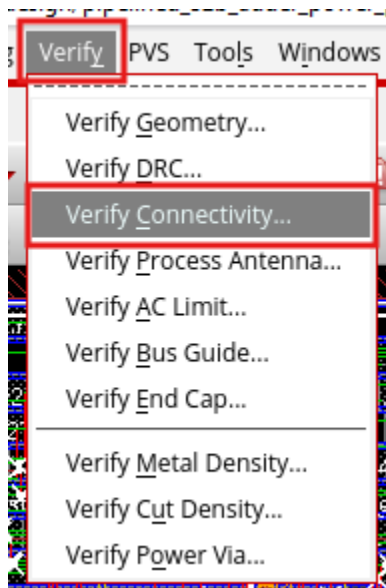


Figure 39: Verify → Verify Connectivity menu.

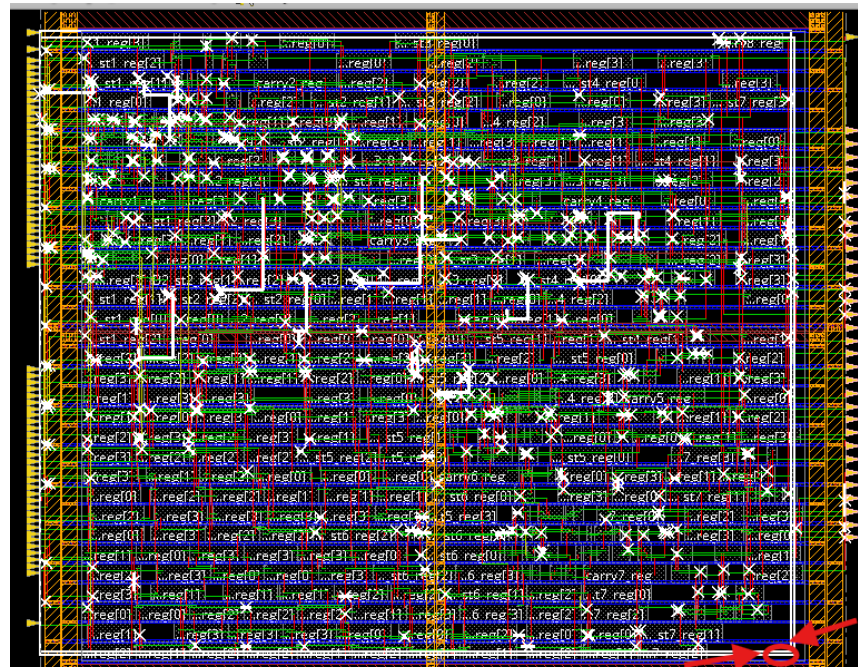


Figure 40: Connectivity errors highlighted on specific nets and cell instances (e.g., carry and reg pins) in the layout.

Part 11: Fixing Geometry Errors and Final Routing

47. Click Route → NanoRoute → Route to run the detailed router, which will also resolve most of the outstanding geometry and connectivity issues left over from placement and CTS.

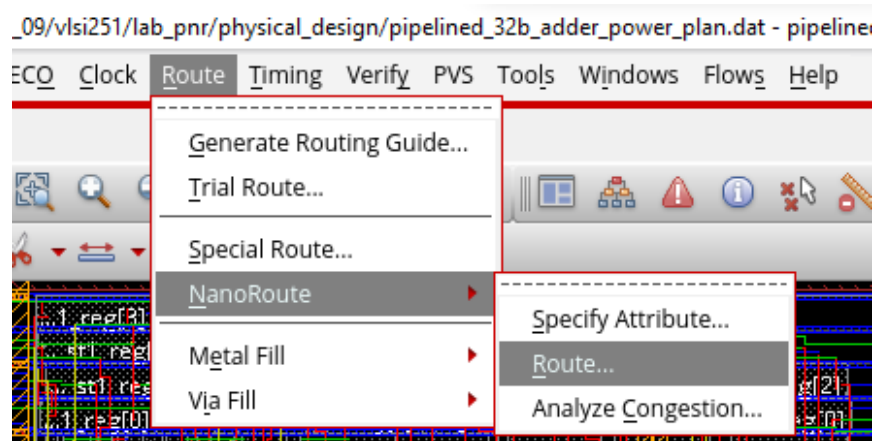


Figure 41: Route → NanoRoute → Route menu.

48. In the NanoRoute window, keep Global Route and Detail Route enabled under Routing Phase, and Fix Antenna enabled under Concurrent Routing Features. Click OK.

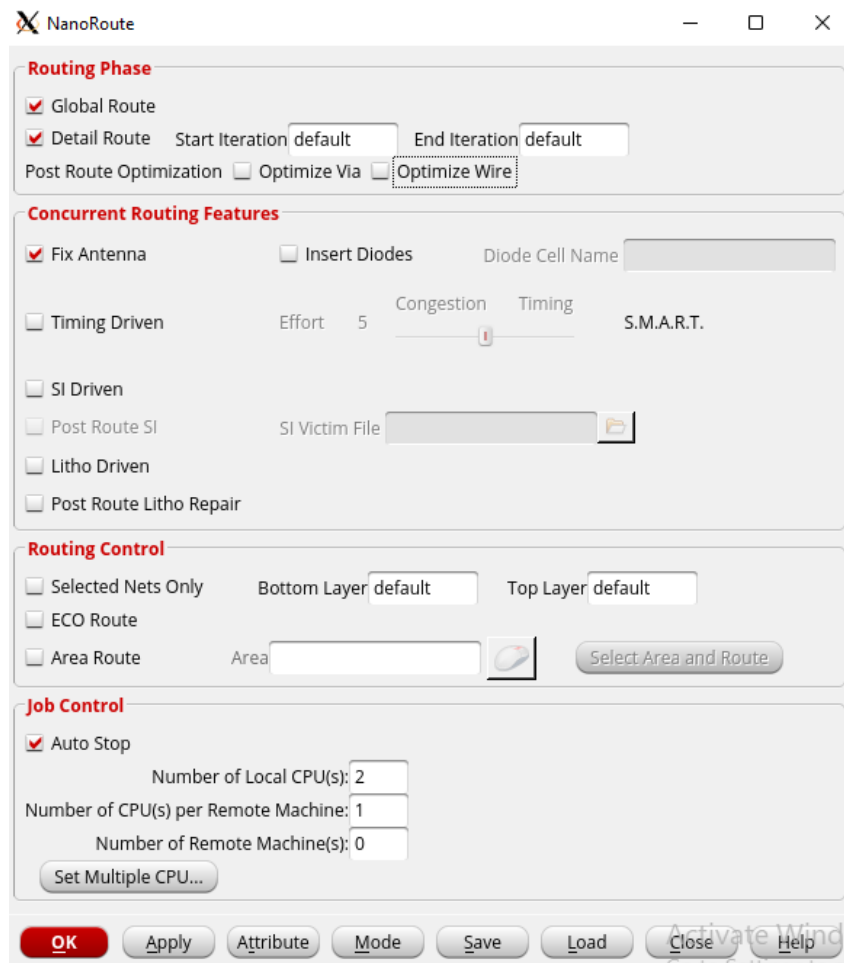


Figure 42: NanoRoute window, configured for a full global-plus-detail routing run with antenna fixing enabled.

49. Once routing completes, re-run Verify → Verify Geometry (and Verify Connectivity). Repeat routing and verification until no violations or warnings remain.

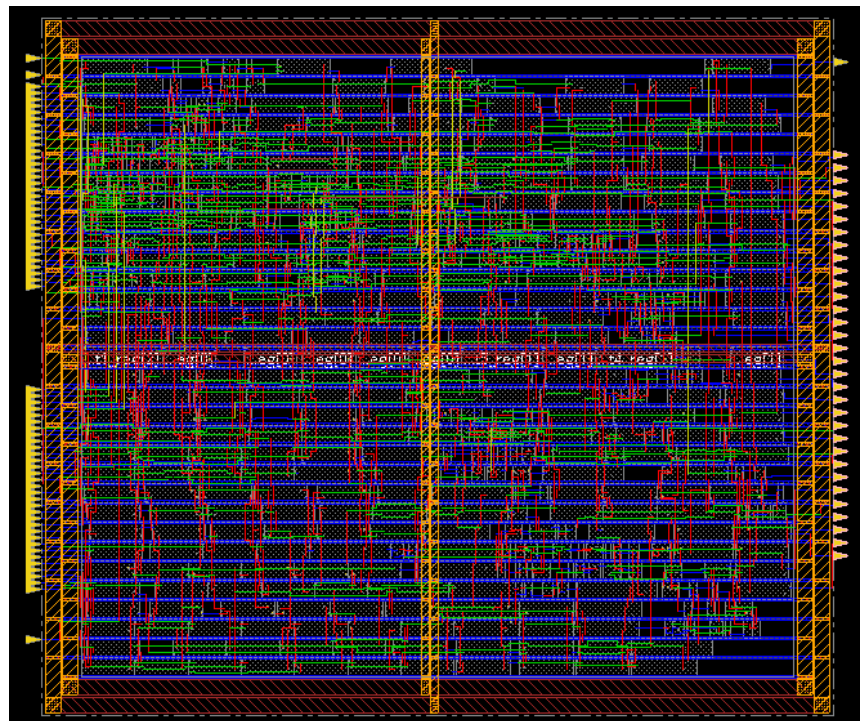


Figure 43: Fully routed design, with no remaining geometry error markers.

```

VERIFY GEOMETRY ..... Cells           : 0 Viols.
VERIFY GEOMETRY ..... SameNet          : 0 Viols.
VERIFY GEOMETRY ..... Wiring           : 0 Viols.
VERIFY GEOMETRY ..... Antenna           : 0 Viols.
VERIFY GEOMETRY ..... Sub-Area : 1 complete 0 Viols. 0 Wrngs.
VG: elapsed time: 0.00
Begin Summary ...
Cells           : 0
SameNet         : 0
Wiring          : 0
Antenna         : 0
Short           : 0
Overlap         : 0
End Summary

Verification Complete : 0 Viols. 0 Wrngs.

*****End: VERIFY GEOMETRY*****
*** verify geometry (CPU: 0:00:00.6 MEM: 0.0M)

```

Figure 44: Verify Geometry summary confirming Verification Complete: 0 Viols. 0 Wrngs.

Part 12: GDSII Export

50. Once the design is fully placed, clock-tree synthesized, routed, and verified with zero violations, click File → Save → GDS/OASIS... to export the final layout.

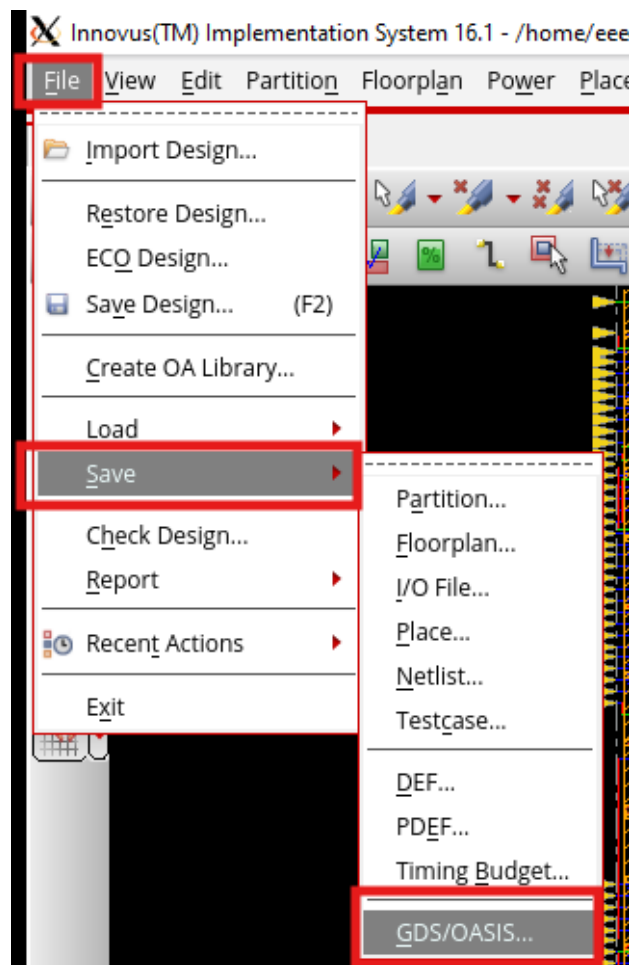


Figure 45: File → Save → GDS/OASIS menu.

51. In the GDS/OASIS Export window, select GDSII/Stream as the Output Format, give the Output File a name (e.g., Adder_32bit_8_stage_final), set the Map File and Library Name, and click OK. This GDSII file is what would be sent to the foundry for fabrication.

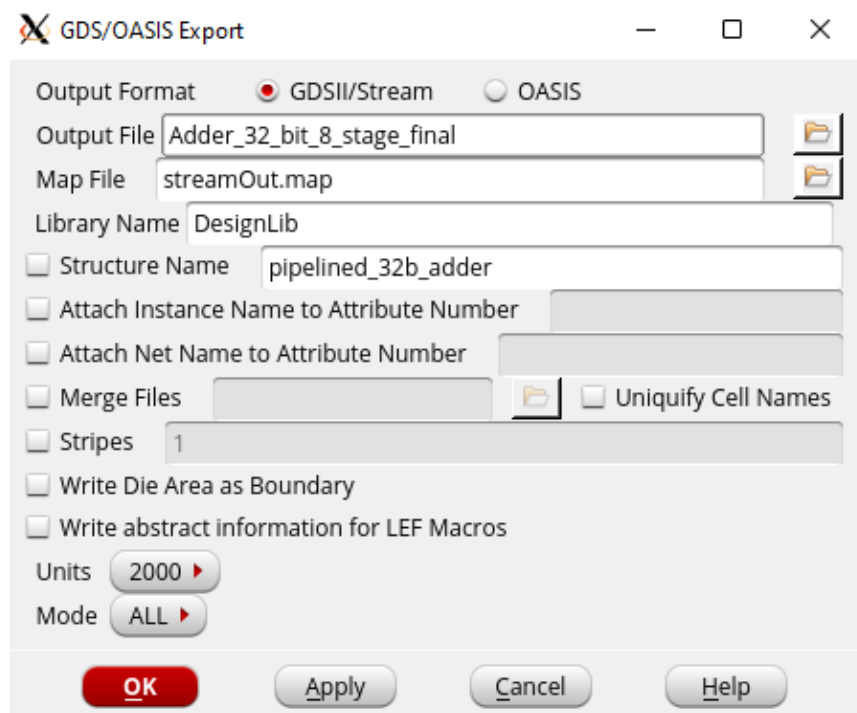


Figure 46: GDS/OASIS Export window, configured to write the final GDSII stream file for the design.

Discussion: From RTL to GDSII

Across Experiments 6 and 8, the same 32-bit pipelined adder has now travelled the full standard-cell design flow: RTL simulation, logic synthesis against a technology library and timing constraints, and finally physical design - floorplanning, power planning, placement, clock tree synthesis, routing, and verification - ending in a GDSII file ready for fabrication. Compare the timing slack reported by Genus after synthesis (Experiment 6) with the pre-CTS and post-CTS slack reported by Innovus in this experiment. Discuss why the numbers differ: synthesis estimates delay using an idealized wireload model, while Innovus computes delay from the actual placed-and-routed geometry, including real wire parasitics and the clock tree's own insertion delay and skew.

Credits

This lab manual was prepared and formatted by:

Originated: EEE, UIU

Sudipto Mondal

Lecturer, Department of Computer Science and Engineering (CSE)

United International University (UIU)

Email: sudiptomondal@cse.uiu.ac.bd